

Documentação Técnica

Credit Risk Modeling - Implantação, Inferência e Retreinamento

Este documento descreve a implementação técnica atual, suas decisões de engenharia, operação local e pontos que ainda exigem evolução antes de uma produção regulada.

Versão 1.0 | Documento de referência operacional

Repository: credit-risk-modeling

Sumário

Sumário	2
1. Objetivo e escopo	3
2. Dados e fonte	3
3. Resultados dos notebooks	3
4. Modelo e contrato de predição	4
5. Arquitetura de implantacao	5
6. Configuração e gestão de dependências	5
7. Operação dos containers	6
8. Inferência em lote	6
9. Retreinamento e promoção	7
10. Monitoramento e Power BI	8
11. Segurança, governança e limites	8
12. Checklist de entrega	9

1. Objetivo e escopo

O projeto estima a probabilidade de inadimplência de solicitações de crédito em um contexto de processamento em lote. Ele cobre a preparação de dados relacionais, análise exploratória, engenharia e seleção de features, treinamento, registro do modelo e uma base operacional para inferência, retreinamento e monitoramento.

O objetivo de engenharia é garantir reprodutibilidade e rastreabilidade: uma predição deve poder ser associada ao batch recebido, ao alias do modelo consultado, a versão efetivamente resolvida no MLflow e ao threshold usado na decisão.

Escopo atual: a inferência e o retreinamento são DAGs manuais no Airflow. A infraestrutura e o esquema de monitoramento existem; a DAG de inferência publica o arquivo pontuado e ainda precisa ser conectada a gravação das facts de predição e de monitoramento no PostgreSQL.

2. Dados e fonte

A base é derivada da competição Home Credit Default Risk, publicada no Kaggle. A licença, os termos de uso e a atribuição da fonte devem ser respeitados em qualquer redistribuição:

<https://www.kaggle.com/c/home-credit-default-risk>.

A tabela central é `application_train.csv`, com uma aplicação atual por `SK_ID_CURR` e a variável alvo `TARGET`. As demais fontes representam histórico de bureau, solicitações anteriores, POS/cash, cartão de crédito e parcelas. Relações um-para-muitos são agregadas antes do join para preservar uma linha por aplicação.

Arquivo	Chave / granularidade	Uso
<code>application_train.csv</code>	<code>SK_ID_CURR</code> ; aplicação atual	Tabela base e origem de <code>TARGET</code>
<code>bureau.csv</code> + <code>bureau_balance.csv</code>	Crédito externo e histórico mensal	Agregação por bureau e depois por cliente
<code>previous_application.csv</code>	Solicitação anterior	Agregação por cliente
<code>POS_CASH_balance.csv</code>	Saldo mensal POS/cash	Agregação por cliente
<code>credit_card_balance.csv</code>	Saldo mensal de cartão	Agregação por cliente
<code>installments_payments.csv</code>	Pagamento de parcela	Agregação por cliente

O notebook 00 grava `data/raw/conjunto_completo.csv`; os splits de treino, validação e teste ficam em `data/interim/`. Dados brutos e intermediários não devem ser versionados no Git.

3. Resultados dos notebooks

A análise exploratória trabalhou com 307.511 aplicações e taxa de inadimplência de 8,07%. A baixa prevalência torna a acurácia inadequada como métrica principal; PR-AUC, recall, KS e ponderação de classe foram priorizados.

A análise observou missingness relevante em variáveis de moradia e scores externos: `BASEMENTAREA_MEDI` apresentou 58,52% de ausências e `EXT_SOURCE_1` 56,38%. Indicadores de ausência devem ser avaliados junto com a imputação.

Os splits estratificados preservaram a taxa de evento: 8,0729% em treino, 8,0728% em validação e 8,0728% em teste. Associações por contrato, gênero e outliers foram tratadas como sinais descritivos para monitoramento, não como regras causais de decisão.

Métrica final em teste	Resultado
Precision	0,2569
Recall	0,4661
F1-score	0,3313
PR-AUC	0,2789
ROC-AUC	0,7807
KS	0,4193
Threshold	0,6500

Esses valores correspondem a avaliação final registrada do modelo campeão. O conjunto de teste foi mantido fora de treinamento, tuning e seleção de threshold.

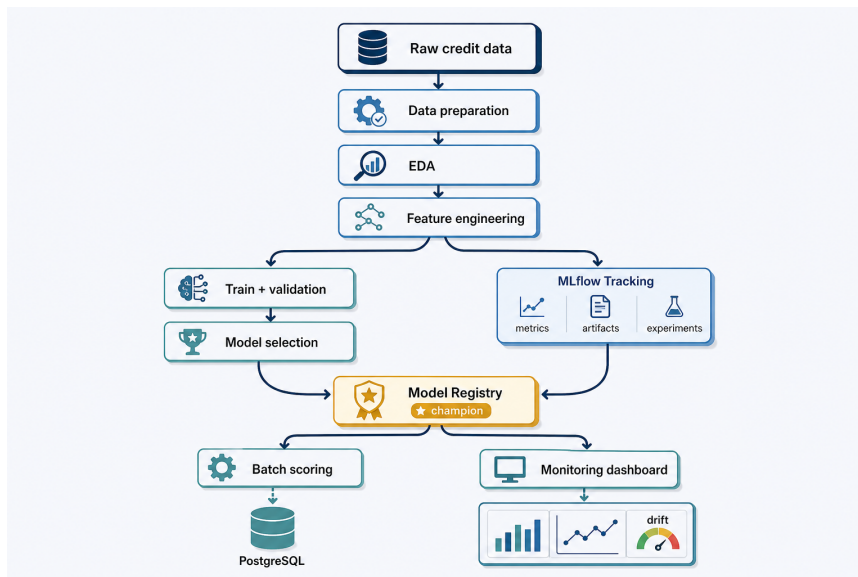
4. Modelo e contrato de predição

O modelo selecionado é um LightGBM binário (GBDT). A configuração manual foi retida porque a otimização com Optuna não apresentou melhor generalização na validação. Os hiperparâmetros ficam versionados em `src/config/model_config.yml`.

O pipeline inclui `StandardScaler` para colunas numéricas e `OrdinalEncoder` para categóricas. Categorias desconhecidas recebem -1, permitindo pontuar batches com novas categorias sem falha de codificação. A ponderação de classe é calculada com o alvo de treino.

A seleção do modelo usa PR-AUC em validação cruzada. O threshold é escolhido no conjunto de validação, maximizando F1-score. O teste é reservado para a avaliação final e para a comparação `champion` versus `challenger` no retreinamento.

Contrato	Definição
Entrada	67 features selecionadas. O módulo aplica feature engineering quando recebe dados brutos; se já receber o contrato completo, apenas reordena as colunas.
Saída	<code>probability_default</code> (double) e <code>default_prediction</code> (inteiro binário).
Registro	MLflow Model Registry: <code>credit-risk-model</code> ; alias operacional: <code>champion</code> .
Rastreabilidade	Assinatura, input example, output example, threshold, métricas e artefatos de interpretabilidade ficam no MLflow.



Fluxo de machine learning do projeto.

5. Arquitetura de implantacao

A implantacao e local e containerizada por Docker Compose. PostgreSQL separa logicamente os bancos de MLflow, Airflow e monitoramento; o MLflow armazena metadados no PostgreSQL e artefatos em volume Docker; Airflow executa as DAGs e acessa dados e codigo por volumes montados.

Servico	Responsabilidade	Porta / persistência
postgres	Banco para MLflow, Airflow e monitoramento	5432 interna; volume postgres_data
mlflow	Tracking, artefatos e Model Registry	http://localhost:5000; volume artifacts_data
database-bootstrap	Cria bancos airflow e monitoring e aplica migration idempotente	Executa e finaliza
airflow-init	Migra metadata do Airflow e cria usuario administrador	Executa e finaliza
airflow-webserver	Interface para disparar e observar DAGs	http://localhost:8080
airflow-scheduler	Agenda e executa tarefas	Servico interno

Os volumes postgres_data, artifacts_data e airflow_logs sobrevivem ao comando de parada normal. Por isso, artefatos do MLflow nao aparecem como arquivos comuns na pasta do repositorio.

6. Configuração e gestão de dependências

pyproject.toml declara as dependências diretas e a faixa de Python suportada. uv.lock fixa a resolução completa, incluindo dependências transitivas; ele deve ser versionado e nunca editado manualmente. Execute uv sync apos alterar o manifesto ou trocar de maquina.

A compatibilidade do modelo e parte do contrato operacional. O campeão atual foi serializado em Python 3.11.7 com scikit-learn 1.9.0, LightGBM 4.4.0, Pandas 2.1.4 e Cloudpickle 2.2.1. O projeto e a imagem do Airflow foram alinhados a essa familia de versoes. Um novo campeão so deve ser promovido apos ser validado no mesmo runtime de inferencia.

Arquivo	Papel
.python-version	Solicita Python 3.11 para ferramentas compatíveis.
pyproject.toml	Manifesto de dependências e configurações de pytest/uv.
uv.lock	Grafo de dependências resolvido e reproduzível.
docker/Dockerfile.airflow	Imagem Airflow 2.11.2 com Python 3.11 e runtime do campeonato.
docker/requirements-airflow.txt	Dependências da imagem operacional, fixadas para carga segura do modelo.
docker-compose.yml	Topologia, volumes, portas, health checks e variáveis dos serviços.

O arquivo .env fica apenas no ambiente local e é ignorado pelo Git. Ele precisa definir, no mínimo: POSTGRES_PASSWORD, AIRFLOW_FERNET_KEY, AIRFLOW_WEBSERVER_SECRET_KEY, AIRFLOW_ADMIN_PASSWORD e MONITORING_KEY_SALT. Opcionalmente define nomes de bancos e usuário administrador. Senhas devem ser URL-safe porque compoem URIs PostgreSQL.

7. Operação dos containers

Pre-requisitos: Docker Desktop, Docker Compose e uv. No Windows, abra o repositório clonado no VS Code e use o terminal PowerShell na raiz do projeto.

Comandos principais:

```
uv sync
docker-compose up -d --build
docker-compose ps
docker-compose logs -f airflow-scheduler
docker-compose down
```

Use `docker-compose down -v` somente quando quiser apagar deliberadamente bancos, artefatos e logs persistidos.

Depois da inicialização, MLflow fica em <http://localhost:5000> e Airflow em <http://localhost:8080>. As credenciais do Airflow são as definidas em .env; elas não devem ser documentadas ou enviadas ao repositório.

Validações operacionais recomendadas: `uv lock --check`, `docker-compose exec -T airflow-scheduler python -m pip check`, `docker-compose exec -T airflow-scheduler airflow dags list-import-errors` e verificação de health checks em `docker-compose ps`.

8. Inferência em lote

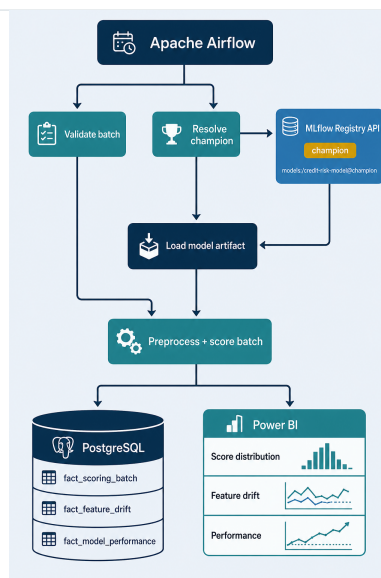
A DAG `credit_risk_batch_inference` é manual (`schedule=None`) e possui três tarefas: `load_data`, `predict` e `save_predictions`. A opção manual é intencional enquanto a fonte operacional de batches não está definida.

O disparo deve informar caminhos vistos de dentro do container. Como a pasta local `data/` está montada em `/opt/airflow/data`, um arquivo local `data/batches/input.parquet` deve ser referenciado como `/opt/airflow/data/batches/input.parquet`.

Configuração de exemplo para o Airflow:

```
{
  "input_path": "/opt/airflow/data/batches/input.parquet",
  "output_path": "/opt/airflow/data/scored/scored_2026_08.parquet"
}
```

Etapa	Comportamento
load_data	Valida existencia e extensao CSV/Parquet e cria um batch_id.
predict	Carrega models:/credit-risk-model@champion, resolve a versao concreta, aplica o contrato de features e adiciona as duas saidas.
save_predictions	Grava primeiro em .staging e publica por move atomico. Arquivos existentes nao sao sobrescritos.



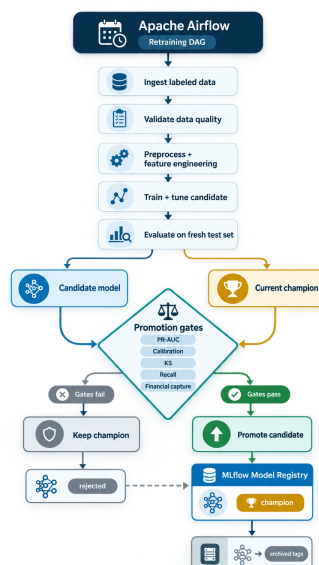
Arquitetura de inferência em lote e monitoramento planejado.

9. Retreinamento e promoção

A DAG `credit_risk_retraining` também é manual e requer um arquivo rotulado com `TARGET`. Ela valida a entrada, cria splits estratificados de treino, validação e teste, treina o challenger, seleciona threshold na validação e compara challenger e champion no mesmo teste mantido fora do treinamento.

A regra implementada de promoção é estrita: challenger PR-AUC deve ser maior que champion PR-AUC. O challenger sempre é registrado; recebe alias challenger e tags de validação. Quando aprovado, o alias champion move para ele e a versão anterior recebe `lifecycle_status=archived`. Quando rejeitado, champion permanece inalterado.

Importante: para uma operação produtiva, substitua o split aleatório atual por janelas temporais e um holdout out-of-time. Antes da promoção, recomenda-se adicionar guardrails de calibração, KS, recall mínimo, captura financeira, qualidade de dados e revisão de fairness.



Arquitetura de retreinamento e promoção do challenger.

10. Monitoramento e Power BI

A migration database/migrations/001_monitoring_star_schema.sql cria o schema monitoring no banco monitoring. A estrutura adota star schema para consumo analítico no Power BI; a dimensao de calendario fica no proprio modelo semantico do Power BI.

Dimensoes: dim_model, dim_feature, dim_segment, dim_data_reference e dim_validation_rule. Facts: fact_scoring_batch, fact_prediction, fact_training_observation, fact_training_feature_profile, fact_feature_monitoring, fact_prediction_monitoring, fact_data_quality_result, fact_directional_expectation_result e fact_model_performance.

O baseline de treino vincula uma versao de modelo a suas features, target e perfil estatistico. Identificadores de aplicacao sao anonimizados com MONITORING_KEY_SALT antes da persistência. A futura carga de predicoes deve registrar versao do modelo, batch, probabilidade, threshold, decisao e alvo observado quando disponivel.

Area	Métricas e validações
Qualidade	Volume, duplicidade, colunas requeridas, tipo, range, regex e valores permitidos.
Drift de features	PSI, mudanca de missingness, quantis numericos, distribuicao categorica e categorias novas.
Drift de predição	PSI da score distribution, media de probabilidade e taxa de crossing do threshold.
Performance	PR-AUC, ROC-AUC, KS, precision, recall, F1, Brier, calibracao e exposicao financeira capturada.
Expectativa direcional	Verifica se a direcao entre uma feature e risco segue uma hipotese aprovada pelo negocio; nao define causalidade.

11. Seguranca, governança e limites

Segredos permanecem em .env e nunca em commits, notebooks, screenshots ou documentacao. Em uma implantacao compartilhada, use um gerenciador de segredos e contas de banco com privilegios minimos.

O alias champion e uma referência mutavel; cada scoring deve persistir a versao resolvida e o run_id, nunca somente o alias. Modelos e artefatos no MLflow devem ser tratados como imutaveis apos registro.

O aviso historico de pathlib no carregamento do artefato atual e uma inconsistencia nao bloqueante do metadata antigo do MLflow. Ele nao equivale aos avisos de desserializacao do scikit-learn, que foram eliminados ao alinhar o

runtime. O artefato registrado não deve ser alterado manualmente.

Antes de uso de crédito real, são necessários controles adicionais de privacidade, autorização, auditoria, explicabilidade aprovada por risco, validação independente e políticas de retenção de dados.

12. Checklist de entrega

1. Executar `uv sync` e `uv lock --check`. 2. Preencher `.env` sem expor segredos. 3. Subir `docker-compose up -d --build`. 4. Confirmar MLflow, Airflow e PostgreSQL saudáveis. 5. Confirmar que o alias `champion` existe no MLflow. 6. Testar uma inferência manual com arquivo de entrada e caminho de saída novo. 7. Verificar schema, assinatura e artefatos do modelo. 8. Para retreinamento, usar dados rotulados e revisar o resultado da comparação antes de qualquer promoção em ambiente regulado.